

제12강

1) Firebase Realtime Database를 이용한 즐겨찾기 등록, 삭제

- [Firebase 내맵]-[제품 카테고리]-[Realtime Database]-[규칙]에서 rules의 read, write을 true로 변경한다.

[src]-[components12] BookPage.jsx

```
import Favorite from './book/Favorite'
...
const BookPage = () => {
  return (
    <Card.Body className='position-relative'>
      <Favorite book={book}/>
      <BookModal book={book}/>
    </Card.Body>
  )
}
```

[src]-[components12]-[book] Favorite.jsx

```
import React, { useContext, useEffect, useState } from 'react'
import { FaRegHeart, FaHeart } from 'react-icons/fa'
import { app } from '../initFirebase'
import { getDatabase, ref, set, onValue, remove } from 'firebase/database'
import { useNavigate } from 'react-router';
import { ModalContext } from '../context/ModalContext';

const Favorite = ({book}) => {
  const db = getDatabase(app);
  const navi = useNavigate();
  const { showAlert } = useContext(ModalContext);
  const [favorite, setFavorite] = useState(false);

  const uid = sessionStorage.getItem('uid');
  const bookRef = `~/favorite/${uid}/${book.isbn}`;
  const onRegister = async() => {
    if(uid) {
      await set(ref(db, bookRef), book);
      showAlert({ show:true, message:'즐거찾기에 등록되었습니다.' });
    } else {
      navi('/login')
    }
  }

  useEffect(()=>{
    onValue(ref(db, bookRef), snapshot=>{
      if(snapshot.exists()){
        setFavorite(true);
      } else {
        setFavorite(false);
      }
    });
  },[]);

  const onRemove = async() => {
    await remove(ref(db, bookRef));
    showAlert({show:true, message:'즐거찾기에서 삭제되었습니다.'});
  }

  return (
    <div className='position-absolute top-0 end-0 me-2 icon-heart'>
      { favorite ? <FaHeart onClick={onRemove}/> : <FaRegHeart onClick={onRegister}/>}
    </div>
  )
}
```

```
.icon-heart {
  cursor: pointer;
  font-size: 25px;
  color: orange;
}
```

2) Firebase Realtime Database를 이용한 즐겨찾기 목록

[src]-[components12] MainRouter.jsx

```
import FavoritePage from './book/FavoritePage'
const MainRouter = () => {
  <Route path='favorite' element={}<FavoritePage/>}</>
}
```

[src]-[components12]-[book] FavoritePage.jsx

```
const FavoritePage = () => {
  const db = getDatabase(app);
  const [books, setBooks] = useState([]);
  const [loading, setLoading] = useState(false);
  const { setConfirm } = useContext(ModalContext);
  const uid = sessionStorage.getItem('uid');

  const callAPI = () => {
    setLoading(true);
    const bookRef = `/${favorite}/${uid}`;
    onValue(ref(db, bookRef), snapshot=>{
      const rows=[];
      snapshot.forEach(row=>{
        rows.push({ key:row.key, ...row.val() });
      });
      setBooks(rows);
    });
    setLoading(false);
  }

  useEffect(()=>{
    callAPI();
  }, []);

  const onRemove = (book) => {
    setConfirm({
      show:true,
      message:`${book.title}' 도서를 삭제하실래요?`,
      action: async()=>{
        const bookRef = `/${favorite}/${uid}/${book.isbn}`;
        await remove(ref(db, bookRef));
      }
    });
  };

  if(loading) return <h1 className='text-center my-5'>로딩중...</h1>
  return (
    <div className='my-5'>
      <h1 className='text-center mb-5'>즐거찾기</h1>
      <Table hover striped bordered>
        <tbody>
          { books.map((book, index)=>
            <tr id={index}>
              <td>{index+1}</td>
              <td>{book.title}</td>
              <td>{parseInt(book.price).toLocaleString()}원</td>
              <td>{book.authors}</td>
              <td><Button onClick={()=>onRemove(book)} variant='outline-danger' size='sm'>삭제</Button></td>
            </tr>
          )}
        </tbody>
      </Table>
    </div>
  )
}
```